

# A Structure-Agnostic Co-Tuning Framework for LLMs and SLMs in Cloud-Edge Systems

Yuze Liu  
Swinburne University of Technology  
Melbourne, Australia  
yuzeliu@swin.edu.au

Yunhan Wang  
Tongji University  
Shanghai, China  
2354180@tongji.edu.cn

Tiehua Zhang\*  
Tongji University  
Shanghai, China  
tiehuaz@tongji.edu.cn

Zhishu Shen  
Wuhan University of Technology  
Wuhan, China  
z\_shen@ieee.org

Cheng Peng  
INFLY TECH (Shanghai) Co., Ltd.  
Shanghai, China  
sophiapeng0426@hotmail.com

Libing Wu  
Wuhan University  
Wuhan, China  
wu@whu.edu.cn

Feng Xia  
RMIT University  
Melbourne, Australia  
feng.xia@rmit.edu.au

Jiong Jin  
Swinburne University of Technology  
Melbourne, Australia  
jiongjin@swin.edu.au

## Abstract

The surge in intelligent applications driven by large language models (LLMs) has made it increasingly difficult for bandwidth-limited cloud servers to process extensive LLM workloads in real time without compromising user data privacy. To solve these problems, recent research has focused on constructing cloud-edge consortia that integrate server-based LLM with small language models (SLMs) on mobile edge devices. Furthermore, designing collaborative training mechanisms within such consortia to enhance inference performance has emerged as a promising research direction. However, the cross-domain deployment of SLMs, coupled with structural heterogeneity in SLMs architectures, poses significant challenges to enhancing model performance. To this end, we propose Co-PLMs, a novel co-tuning framework for collaborative training of large and small language models, which integrates the process of structure-agnostic mutual learning to realize knowledge exchange between the heterogeneous language models. This framework employs distilled proxy models (DPMs) as bridges to enable collaborative training between the heterogeneous server-based LLM and on-device SLMs, while preserving the domain-specific insights of each device. The experimental results show that Co-PLMs outperforms state-of-the-art methods, achieving average increases of 5.38% in Rouge-L and 4.88% in EM. Our code has been released at <https://github.com/papercode-DFL/Co-PLMs>.

## CCS Concepts

• **Computing methodologies** → **Distributed artificial intelligence**.

\*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.  
WWW '26, Dubai, United Arab Emirates.  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792682>

## Keywords

Large Language Models, Small Language Models, Knowledge Transfer, Co-Tuning

### ACM Reference Format:

Yuze Liu, Yunhan Wang, Tiehua Zhang, Zhishu Shen, Cheng Peng, Libing Wu, Feng Xia, and Jiong Jin. 2026. A Structure-Agnostic Co-Tuning Framework for LLMs and SLMs in Cloud-Edge Systems. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3774904.3792682>

## 1 Introduction

The rapid development of large language models (LLMs) has sparked significant interest in their application to mobile devices and Web-of-Things (WoT), enabling intelligent services such as image recognition [8], natural language understanding [19], and health monitoring [21], etc. LLMs are typically deployed on remote cluster servers to accommodate the significant computational resources required for inference [12] and the need for ongoing updates via fine-tuning [35]. However, the high volume of inference requests from a large number of edge devices creates communication bottlenecks, while the data demands of fine-tuning raise growing concerns about privacy leakage [22]. As a result, relying solely on server-based LLMs is neither efficient nor practical for real-world IoT and WoT applications [3].

To address these issues, recent research has focused on deploying small language models (SLMs), such as TinyLlama [40], Gemma [26] and MobileVLM [5], on resource-constrained devices, and adopting federated learning (FL) to collaboratively train on-device SLMs for continual enhancement of model performance while preserving local data privacy [2, 35, 38]. However, existing methods largely overlook the potential collaboration between SLMs and LLMs, failing to fully combine the strong generalization capabilities of LLMs with the efficiency of SLMs for on-device deployment. Therefore, designing an effective collaboration framework between on-device SLMs and server-based LLMs has emerged as a promising direction for future research [3, 7, 33].

Despite its potential, developing an effective collaboration framework for practical cloud-edge systems still faces two key challenges:

- **Model Heterogeneity.** In real-world scenarios, IoT and WoT devices exhibit diverse computational and communication capabilities due to varying hardware specification, necessitating the deployment of SLMs that differ in model size and model type [28]. This disparity renders existing FL methods, such as FedLoRA [44] and FedAP [15], which employ aggregation-based collaborative training, incapable of accommodating model heterogeneity [37].
- **Domain Heterogeneity.** Due to personalized usage patterns and diverse deployment environments, the local datasets on devices often exhibit domain-specific biases [25], which can lead to model performance degradation during the collaborative learning process [39].

To address these challenges, we propose a domain-aware heterogeneous co-tuning framework, termed Co-PLMs, which enables collaborative training between SLMs and LLMs within a cloud-edge system. This framework is composed of three key components, including initialization of the proxy model through model distillation from the server-based LLM, domain-specific tuning (DST) of the distilled proxy model (DPM) to capture domain biases within local datasets on devices and structure-agnostic mutual learning (SAML) between DPMs and language models to enable efficient bidirectional knowledge exchange. At the beginning of the collaborative learning process, the DPM is distilled from the server-based LLM and then distributed to each device, where domain adapters are integrated. During each collaborative training round, every device locally trains its DPM through DST to preserve domain-specific insights and performs SAML between the DPM with its local SLM to enable bidirectional knowledge transfer. Subsequently, the tunable parameters of the DPMs from all devices are uploaded to the server and aggregated to update the server-side DPM. The server then performs SAML between the LLM and the updated DPM to further update the LLM. Finally, the tunable parameters of the server-side DPM are redistributed to each device with the aim of propagating knowledge from the LLM.

The main advantages of our proposed framework are two folds: 1) DPMs act as bridges that enable collaborative training between heterogeneous language models within cloud-edge systems, where the exchange of DPM parameters facilitates effective knowledge sharing while preserving local data privacy. 2) Performing DST and SAML preserves domain-specific biases for each device while reducing computational costs through a parameter-efficient fine-tuning strategy. Furthermore, the communication overhead in Co-PLMs is significantly alleviated by exchanging only the tunable parameters of the DPM with the server during the collaborative learning process.

In summary, the main contributions of this work are:

- We propose Co-PLMs, a novel co-tuning framework for collaborative training of large and small language models within cloud-edge system. In Co-PLMs, knowledge transfer between heterogeneous on-device SLMs and the server-based LLM relies on their respective DPMs, which act as bridges enabling bidirectional knowledge exchange between edge devices and the cloud server through SAML. Meanwhile, DST of local

DPMs helps preserve domain-specific insights during the collaborative learning process.

- We design SAML, which incorporates bidirectional token alignment to address tokenizer mismatches and an logits pooling strategy to solve the problem divergence singularities during mutual learning, thereby enabling effective bidirectional knowledge transfer between heterogeneous language models.
- We conduct comprehensive experiments on the proposed framework using various publicly available language models across two multi-domain datasets, SNI and MMLU, along with a detailed analysis of communication overhead. Experimental results demonstrate that the proposed Co-PLMs significantly outperforms state-of-the-art baselines on both datasets while maintaining high communication efficiency throughout the collaborative learning process.
- We have released our source code publicly at <https://github.com/papercode-DFL/Co-PLMs> to facilitate further research and development in this field.

## 2 Related Work

### 2.1 LLMs Deployment at the Edge

Traditional LLM deployment is based on centralized cloud servers, which provide sufficient computational and memory resources but suffer from network-dependent latency, high energy consumption, and privacy risks. In parallel, deploying full-scale LLMs directly on edge devices is also impractical, as their limited computation and memory capacity hinder real-time and high-quality inference. To address the aforementioned challenges, several recent studies have proposed collaborative frameworks for LLM deployment at the edge. Specifically, cloud-edge collaboration-based LLM frameworks, such as CE-CoLLM [18] and Edge-LLM [2], offload part of the computation to the cloud while retaining latency-critical tasks at the edge through techniques like early exits, asynchronous context upload, and adaptive quantization. PerLLM [36] further extends this direction by modeling inference scheduling as a multi-armed bandit problem under dynamic cloud-edge conditions. Edge-side optimization-based LLM frameworks, such as Edge-LLM [38] and LLMEdge [24], directly address computation and memory bottlenecks in constrained devices. They exploit unified compression, adaptive layer tuning, and hardware-aware scheduling to improve efficiency, while incorporating quantization and lightweight APIs to support scalable and privacy-preserving deployment in IoT devices.

In summary, existing approaches can be broadly categorized into edge-side optimization techniques and collaborative frameworks between edge and cloud. These insights motivate the design of our framework, which introduces the SLM deployed at the edge, and further enhances lightweight, collaborative, and privacy-preserving deployment of LLMs at the edge.

### 2.2 Federated Learning for LLMs

In edge scenarios, clients are often constrained by limited computational costs and communication resources, in addition to stringent data privacy requirements. Federated learning for LLMs addresses these issues by enabling collaborative, cross-domain model

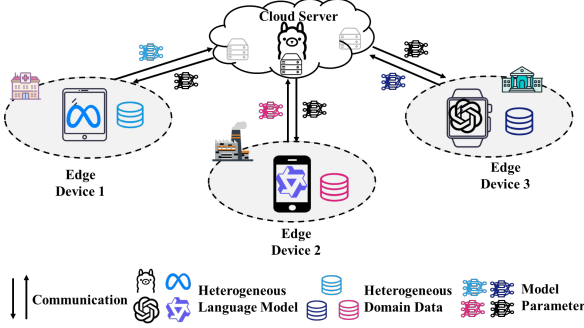


Figure 1: The architecture of the cloud-edge system.

adaptation, allowing clients to personalize models with their local data while keeping it private, and further reducing resource demands through parameter-efficient fine-tuning (PEFT). Recent works in PEFT-based federated learning, such as Federated Prompt Tuning[45], FATE-LLM[4], and FedPETuning[44], leverage adapters, prompts and LoRA to minimize communication overhead efficiently. Notably, FedPFT [23] constructs layer-wise sub-models with neuron-level alignment, FedLoRA[31] decomposes layers into shared and client-specific low-rank matrices with alternating training, and FedPETuning evaluates adapters, LoRA, and prefix tuning for lightweight updates under resource constraints. Furthermore, mutual knowledge transfer, as employed in FedCoLLM[9] and FedMKT[10], introduces LLM deployed at the server and facilitates both server-sides LLMs and client-sides SLMs by utilizing adapters, knowledge distillation, and token alignment.

The aforementioned federated learning approaches either focus on efficient parameter updates or on mutual knowledge transfer between server and clients. Despite their effectiveness, challenges remain in further reducing communication overhead and supporting heterogeneous models with cross-domain setting, but these considerations are fully addressed in our proposed framework.

### 3 Preliminaries

**Problem Definition** As illustrated in Figure 1, we consider an cloud-edge system consisting of a central server and a set of devices, denoted by  $\mathcal{D} = \{d_i\}_{i=1}^N$ , where  $N$  represents the total number of devices. Each device hosts a SLM  $m_i \in \mathcal{M}$ , while the server maintains a LLM denoted by  $M$ . Each SLM and the LLM possesses its own model structure, and the model-type mapping function is defined as  $f_{mt} : \{M\} \cup \mathcal{M} \rightarrow \mathcal{T}$ .  $\mathcal{T}$  denotes the set of model structure types, encompassing various language model architectures such as GPT, Llama, and others. Each device  $d_i$  and server owns a local private dataset denoted as  $D_i^d$  and  $D^s$ , respectively. Specifically, local datasets among device are domain heterogeneity. Specifically, the local datasets across devices exhibit domain heterogeneity.

The server and devices collectively aim to improve the performance of the LLM and SLMs through collaborative learning while preserving data privacy. The collaborative learning process encompasses several rounds of training, in which each device uploads the locally trained model parameters to the server for model aggregation. The updated model parameters are then distributed back to the devices. The objective function to be minimized for the entire

collaborative learning process is defined as:

$$\min_{\theta(M), \{\theta(m_i)\}_{m_i \in \mathcal{M}}} \mathcal{L}(\theta(M), \theta(m_1), \dots, \theta(m_N); D^s, D_1^d, \dots, D_N^d) \quad (1)$$

where  $\theta(M)$  and  $\{\theta(m_i)\}_{m_i \in \mathcal{M}}$  denote the trainable parameters of the server and the devices, respectively. Furthermore, we assume that the  $N$  devices and the server perform the same question-answering (QA) task and employ heterogeneous model structure types, denoted by  $|\mathcal{T}| > 1$ .

**Low-rank Adaptation** LoRA [16] decomposes the update of the weight matrix  $W$  into the product of two low-rank matrices, under the assumption that fine-tuning updates are low-rank [1]. Here,  $W$  denotes the weight matrix of a linear layer in the model. For instance, in Transformer architectures, it may correspond to the query ( $W_q$ ), key ( $W_k$ ), value ( $W_v$ ) in the self-attention layer. Specifically, LoRA has the following mathematical form:

$$W^* = W_0 + \Delta W = W_0 + BA \quad (2)$$

where  $W^*, W_0 \in \mathcal{R}^{n \times m}$ ,  $B \in \mathcal{R}^{n \times r}$ , and  $A \in \mathcal{R}^{r \times m}$ , with  $r \ll \min(n, m)$ .  $W_0$  denotes the pre-trained weight matrix, which remains frozen during the fine-tuning process, while  $B$  and  $A$  are trainable parameters. Owing to its training efficiency, LoRA has emerged as one of the most widely adopted parameter-efficient fine-tuning methods for training large language models [17]. The LoRA fine-tuning process for language models can be formulated as follows:

$$\tilde{m} = f_{lora}(\phi_{lora}(m); m) \quad (3)$$

where  $\phi_{lora}(m)$ ,  $m \in \{M\} \cup \mathcal{M}$  denotes the trainable parameters during the LoRA fine-tuning process, and  $\tilde{m}$  represents the local language model after fine-tuning.

## 4 Methodology

An overview of the Co-PLMs architecture is presented in Figure 2. In this section, we first describe the model distillation process from the LLM, which initializes the DPM. Next, DST is performed on the DPMs to capture domain biases, followed by SAML, which enables bidirectional knowledge exchange between DPMs and language models on edge devices. On the server side, SAML is conducted between the updated DPM, aggregated from uploaded parameters, and the server-based LLM. Finally, the server distributes the parameters of its DPM to the devices to propagate knowledge throughout the system.

### 4.1 Initialization of Distilled Proxy Model

Since the server-based LLM and the device-based SLMs have heterogeneous model structures, traditional FL approaches that directly aggregate local models for collaborative training are not applicable [28]. Previous works have adopted proxy models that serve as bridges to facilitate knowledge transfer across models with heterogeneous architectures [9, 23]. However, existing approaches fail to account for the resource constraints of devices and to fully leverage the capabilities of the server-based LLM. To address these challenges, we adopt a knowledge distillation method, MiniLLM [11], to generate a Transformer-based DPM from the server-based LLM at the beginning of the collaborative learning process. The distillation process used to initialize the DPMs can be formulated as

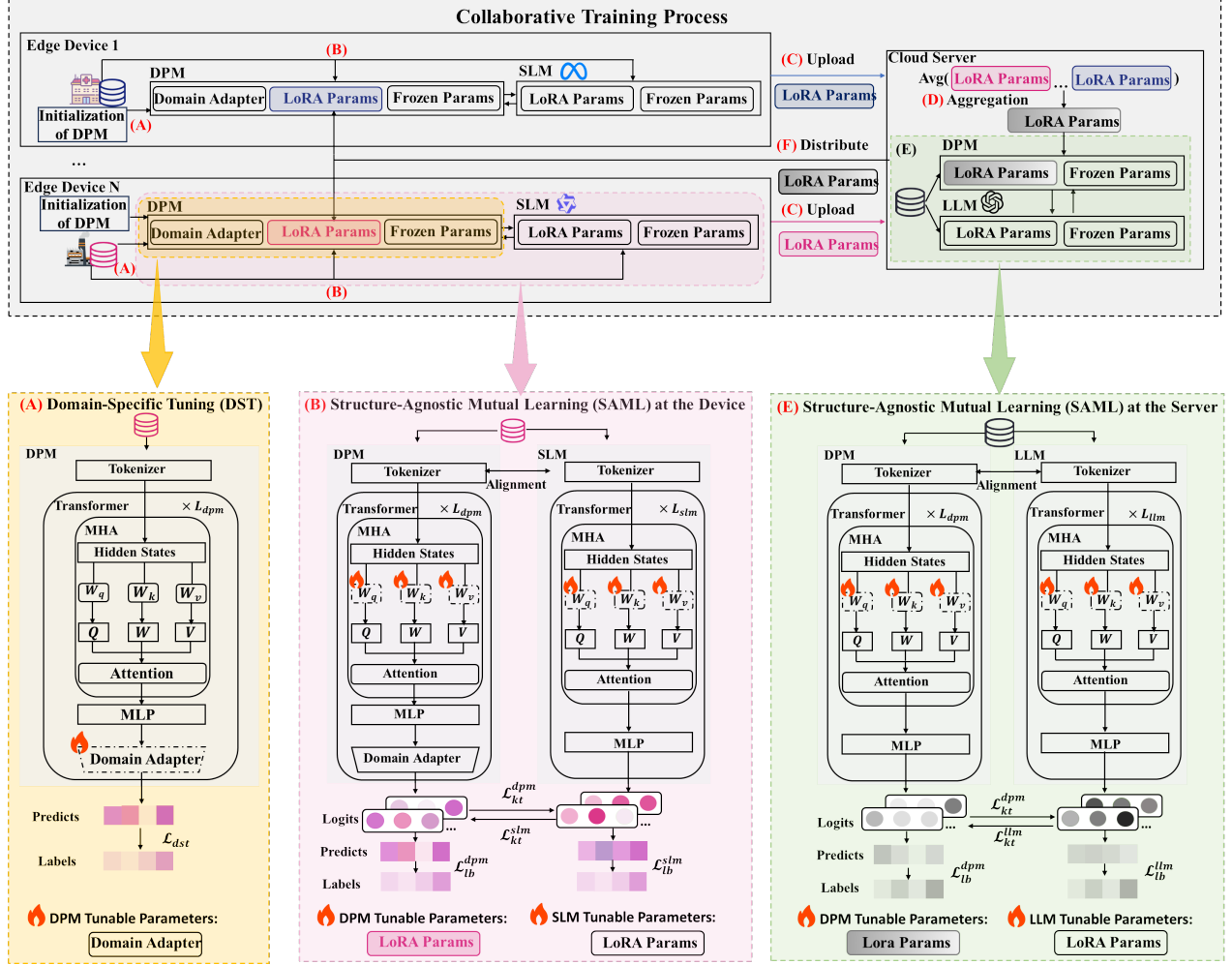


Figure 2: The overview of our proposed framework.

follows:

$$m^p = f_{kd}(M) \quad (4)$$

where  $m^p$  denotes the DPM. After the distillation process is completed,  $m^p$  is distributed to each device to initialize the local DPMs, denoted as  $m_i^p$ ,  $i \in [1, N]$ .

## 4.2 Domain-Specific Tuning

To mitigate the performance degradation caused by domain heterogeneity in the local datasets of devices, we design a domain adapter for the DPM and propose a DST process to preserve its domain-dependent knowledge. We equip each Transformer layer of the DPM with a domain-aware adapter to extract domain-specific knowledge from the hidden representations of that layer. The domain adapter can be implemented using different neural architectures, such as a simple linear projection that maps the input to a lower-dimensional space or a multi-layer perceptron (MLP) that applies a sequence of nonlinear transformations. In this work, we employ a two-layer MLP with GeLU activation [14] as the domain adapter.

As illustrated in part (A) of Figure 2, during the DST process, only the domain adapter is trained, while all other parameters in the

DPM remain frozen. The training is conducted through supervised fine-tuning on local datasets, which can be formulated as follows:

$$\min_{\phi_{da}(m_i^p)} \mathcal{L}_{dst}(\phi_{da}(m_i^p); D_i^d) \quad (5)$$

where  $\mathcal{L}_{dst}(\cdot)$  denotes the loss function used in the DST process, and  $\phi_{da}(m_i^p)$ ,  $i \in [1, N]$  represents the tunable parameters in DPMs during DST. The process of DST can be defined as  $f_{dst}(m_i^p)$ .

## 4.3 Structure-Agnostic Mutual Learning

To enable collaborative training under heterogeneous model architectures, we propose SAML, which leverages DPMs as bridges to facilitate bidirectional knowledge transfer between the devices and the server. As illustrated in parts (B) and (C) of Figure 2, SAML comprises pairs of a DPM and a language model that exchange knowledge by introducing a knowledge-transfer loss applied to their output logits.

**Bidirectional Token Alignment** A significant challenge in computing the mutual knowledge-transfer loss lies in the mismatch between the tokenizers of different language models. Take Qwen

and Llama as an example. Consider the sentence “I utilize the map to travel.” When using the Qwen tokenizer, the sentence is segmented into the following tokens: [‘I’, ‘utilize’, ‘the’, ‘map’, ‘to’, ‘travel’]. In contrast, the Llama tokenizer segments it as [‘I’, ‘util’, ‘ize’, ‘the’, ‘map’, ‘to’, ‘travel’].

To address this issue, we employ a minimum edit distance algorithm with dynamic programming to construct token mappings between the two language models [10]. Using Qwen and Llama as illustrative examples, the token mapping aligns each Llama token with its closest corresponding Qwen token (e.g., ‘utilize’ for ‘util’). The bidirectional token alignment is formulated as  $f_{dpm \rightarrow lm}(Y^p)$  and  $f_{lm \rightarrow dpm}(Y^l)$ , where  $(Y^p)$  and  $(Y^l)$  denote the output logits of the DPM and the language model, respectively.

**Output Logits Pooling for Knowledge Transfer Loss** Previous studies have pointed out that computing knowledge transfer loss typically relies on distribution-based methods such as Kullback–Leibler (KL) divergence [42, 43]. However, the large vocabulary size leads to sparse output distributions, which can cause divergence singularities [6]. To solve this problem, we design an output logits pooling mechanism. Concretely, given the output logits  $Y = \{y_i\}_{i=1}^S$  with sequence length is  $S$ , each token vector  $y_i \in \mathcal{R}^V$  is pooled into a reduced representation  $\tilde{y}_i \in \mathcal{R}^{K+1}$  by preserving its Top-K largest components and aggregating the remaining ones into a single value. The pooling process can be formulated as follows:

$$\tilde{y} = f_{pool}(y) \quad (6)$$

Furthermore, the knowledge transfer loss is defined as:

$$\mathcal{L}_{kt}(Y, Y') = \sum_{i=1}^S KLD(y_i, y'_i) \quad (7)$$

$S = \min(S_1, S_2)$ , where  $S_1$  and  $S_2$  denote the sequence lengths of the output logits  $Y$  and  $Y'$ , respectively. During the process of SAML, the overall loss function  $\mathcal{L}_{\phi_{lora}(m^p)}^{dpm}$  for the DPM can be formulated as follows:

$$\mathcal{L}_{\phi_{lora}(m^p)}^{dpm} = \alpha \mathcal{L}_{kt}^{dpm}(f_{pool}(f_{lm \rightarrow dpm}(Y^l)), f_{pool}(Y^p)) + (1 - \alpha) \mathcal{L}_{lb}^{dpm} \quad (8)$$

where  $\mathcal{L}_{lb}^{dpm}$  denotes the supervised finetuning loss. To achieve efficient fine-tuning in the cloud-edge system, SAML adopts a LoRA-based fine-tuning strategy. Similarly, the objective loss function  $\mathcal{L}_{\phi_{lora}(m)}^{lm}$  for the corresponding language model in each input model pair within SAML can be computed as

$$\mathcal{L}_{\phi_{lora}(m)}^{lm} = \beta \mathcal{L}_{kt}^{lm}(f_{pool}(f_{dpm \rightarrow lm}(Y^p)), f_{pool}(Y^l)) + (1 - \beta) \mathcal{L}_{lb}^{lm} \quad (9)$$

where  $m \in \{M\} \cup \mathcal{M}$ .  $\alpha$  and  $\beta$  are the hyper-parameters that control the proportion of knowledge that from data or from the other model. The knowledge transfer between the DPMs and local language models is bidirectional, and both are trained on the private datasets in the device. The process of SAML can be defined as  $f_{saml}(m^p, m)$ .

#### 4.4 Overall Training Process

The full collaborative training process of Co-PLMs is described in Algorithm 1. The workflow of Co-PLMs proceeds as follows:

- (1) At the beginning of the collaborative learning process, the DPM is distilled from the server-based LLM. The server then

distributes the DPM to each device, where domain adapters are subsequently integrated into it.

- (2) In the  $t$ -th communication round, each device performs DST and SAML sequentially and then uploads the trainable LoRA parameters of its DPM to the server.
- (3) The server aggregates the uploaded LoRA parameters by averaging them to update its DTM and then performs SAML. Afterwards, it distributes the updated LoRA parameters of the DTM to each device.
- (4) Each device receives the LoRA parameters from the server and updates its local DPM accordingly.

---

#### Algorithm 1 Co-PLMs

---

**Input:** The set of devices  $\mathcal{D}$ , the number of device  $N$ , the number of communication round  $T$ , the LLM in the server  $M$ , and the set of SLMs in the devices  $\mathcal{M}$

- 1: Initialize the DPM  $m_s^p(1)$  at the server by Eq.4
  - 2: Distribute the DPM to each device  $\{m_i^p(1)\}_{i=1}^N \leftarrow m_s^p(1)$  and insert with domain adapters
  - 3: **for** each round  $t \in [1, 2, \dots, T]$  **do**
  - 4:   // Device side.
  - 5:   **for** each device  $d_i \in \mathcal{D}$  (in parallel) **do**
  - 6:      $\tilde{m}_i^p(t) \leftarrow f_{dst}(m_i^p(t))$    ▷ Domain-specific tuning
  - 7:     Token alignment between  $\tilde{m}_i^p(t)$  and  $m_i(t)$
  - 8:      $\hat{m}_i^p(t), m_i(t+1), \phi_{lora}(\hat{m}_i^p(t)) \leftarrow f_{saml}(\tilde{m}_i^p(t), m_i(t))$  ▷ Structure-agnostic mutual learning at the device
  - 9:     upload  $\phi_{lora}(\hat{m}_i^p(t))$  to the server
  - 10:   **end for**
  - 11:   // Server side.
  - 12:    $\phi_{lora}(m_s^p(t)) = \frac{1}{N} \sum_{i=1}^N \phi_{lora}(\hat{m}_i^p(t))$  ▷ Model parameters aggregation
  - 13:   Update  $m_s^p(t)$  by  $\phi_{lora}(m_s^p(t))$
  - 14:    $m_s^p(t+1), M(t+1), \phi_{lora}(m_s^p(t+1)) \leftarrow f_{saml}(m_s^p(t), M(t))$  ▷ Structure-agnostic mutual learning at the server
  - 15:   Distribute  $\phi_{lora}(m_s^p(t+1))$  to each device
  - 16:   // Device side.
  - 17:   **for** each device  $d_i \in \mathcal{D}$  (in parallel) **do**
  - 18:      $\phi_{lora}(m_i^p(t+1)) \leftarrow \phi_{lora}(m_s^p(t+1))$
  - 19:     Update  $m_i^p(t+1)$  by  $\phi_{lora}(m_i^p(t+1))$
  - 20:   **end for**
  - 21: **end for**
- 

## 5 Experiment

### 5.1 Experiment Setup

We assume an cloud-edge system comprising one cloud server and  $N = 3$  edge devices to evaluate Co-PLMs using various publicly available language models. The details are as below:

**Models** To evaluate Co-PLMs, we consider a setup comprising, GPT-J-6B [27], deployed on the cloud server, and three SLMs, Bloom-1.1B [30], Llama 2-1.3B [32], and Qwen2.5-1.5B [34], deployed on the edge devices.

**Datasets** We evaluate the performance of our framework on two multi-domain question-answering (QA) datasets, SNI [29] and MMLU [13]. SNI encompasses data from 33 domains, with each

**Table 1: Summary of the comparison results of Co-PLMs on the SNI and MMLU datasets. The best results are highlighted in bold.**

| Dataset | DDS             | Model Structures on the Devices | Method     | Device-1 (Bloom-1.1B) |             | Device-2 (Llama2-1.3B) |             | Device-3 (Qwen2.5-1.5B) |             | Server (GPT-J-6B) |             |
|---------|-----------------|---------------------------------|------------|-----------------------|-------------|------------------------|-------------|-------------------------|-------------|-------------------|-------------|
|         |                 |                                 |            | Rouge-L               | EM          | Rouge-L                | EM          | Rouge-L                 | EM          | Rouge-L           | EM          |
| SNI     | $\lambda = 0.1$ | -                               | Standalone | 39.9                  | 29.6        | 38.7                   | 24.1        | 43.7                    | 31.2        | 53.5              | 37.3        |
|         |                 | Homogeneous                     | FedLoRA    | 33.7                  | 23.3        | 40.4                   | 28.2        | 44.7                    | 29.2        | -                 | -           |
|         |                 |                                 | FedAP      | 28.2                  | 22.1        | 16.3                   | 12.1        | 39.8                    | 28.9        | -                 | -           |
|         |                 |                                 | Ours       | 36.8                  | 24.5        | 40.8                   | 29.4        | <b>54.8</b>             | 37.8        | 58.9              | 37.8        |
|         |                 | Heterogeneous                   | FedCoLLM   | 32.6                  | 21.7        | 38.2                   | 26.4        | 48.7                    | 29.8        | 51.4              | 37.2        |
|         |                 |                                 | FedMKT     | 36.7                  | 24.3        | 41.4                   | 27.8        | 52.8                    | 34.3        | 54.7              | 35.8        |
|         |                 |                                 | Ours       | <b>42.5</b>           | <b>31.7</b> | <b>41.8</b>            | <b>30.2</b> | 54.3                    | <b>39.2</b> | <b>59.2</b>       | <b>39.5</b> |
|         | $\lambda = 1$   | -                               | Standalone | 41.2                  | 27.0        | 44.6                   | 23.5        | 38.2                    | 21.9        | 58.6              | 34.1        |
|         |                 | Homogeneous                     | FedLoRA    | 37.9                  | 24.6        | 48.9                   | 28.1        | 45.8                    | 29.6        | -                 | -           |
|         |                 |                                 | FedAp      | 33.6                  | 20.3        | 23.3                   | 14.4        | 44.6                    | 26.8        | -                 | -           |
|         |                 |                                 | Ours       | 39.1                  | 24.8        | 49.4                   | 28.1        | 47.1                    | 29.9        | 57.6              | 33.8        |
|         |                 | Heterogeneous                   | FedCoLLM   | 38.4                  | 21.8        | 46.4                   | 24.8        | 38.8                    | 23.4        | 54.2              | 32.9        |
|         |                 |                                 | FedMKT     | 39.2                  | 27.1        | 47.6                   | 24.3        | 41.2                    | 26.8        | 56.1              | 32.8        |
|         |                 |                                 | Ours       | <b>44.9</b>           | <b>30.2</b> | <b>51.5</b>            | <b>29.7</b> | <b>50.7</b>             | <b>34.2</b> | <b>59.7</b>       | <b>37.8</b> |
| MMLU    | $\lambda = 0.1$ | -                               | Standalone | 44.0                  | 31.5        | 51.6                   | 23.5        | 49.7                    | 33.5        | 56.8              | 36.6        |
|         |                 | Homogeneous                     | FedLoRA    | 41.2                  | 27.3        | 45.6                   | 20.0        | 54.2                    | 35.5        | -                 | -           |
|         |                 |                                 | FedAp      | 39.1                  | 24.5        | 29.8                   | 19.2        | 49.1                    | 34.0        | -                 | -           |
|         |                 |                                 | Ours       | 44.0                  | 31.8        | 52.1                   | 27.1        | 50.7                    | 36.1        | 55.7              | 34.8        |
|         |                 | Heterogeneous                   | FedCoLLM   | 41.2                  | 29.4        | 48.5                   | 21.6        | 51.1                    | 32.7        | 51.8              | 34.2        |
|         |                 |                                 | FedMKT     | 42.8                  | 30.1        | 48.3                   | 27.2        | 53.7                    | 34.5        | 55.7              | 36.9        |
|         |                 |                                 | Ours       | <b>44.3</b>           | <b>31.0</b> | <b>52.9</b>            | <b>28.5</b> | <b>57.2</b>             | <b>39.4</b> | <b>60.4</b>       | <b>38.0</b> |
|         | $\lambda = 1$   | -                               | Standalone | 43.8                  | 24.8        | 40.1                   | 23.5        | 41.4                    | 20.6        | 52.7              | 29.7        |
|         |                 | Homogeneous                     | FedLoRA    | 42.3                  | 22.8        | 41.5                   | 26.0        | 47.8                    | 26.5        | -                 | -           |
|         |                 |                                 | FedAp      | 40.1                  | 20.6        | 42.1                   | 17.8        | 48.2                    | 21.6        | -                 | -           |
|         |                 |                                 | Ours       | 44.7                  | 26.3        | 44.3                   | 27.5        | 51.2                    | 27.7        | 58.1              | 29.8        |
|         |                 | Heterogeneous                   | FedCoLLM   | 44.8                  | 27.1        | 47.5                   | 26.9        | 48.6                    | 20.9        | 51.7              | 28.6        |
|         |                 |                                 | FedMKT     | 46.1                  | 26.8        | 46.8                   | 27.1        | 49.7                    | 28.5        | 53.8              | 30.1        |
|         |                 |                                 | Ours       | <b>47.2</b>           | <b>29.1</b> | <b>48.2</b>            | <b>29.4</b> | <b>53.2</b>             | <b>33.7</b> | <b>58.9</b>       | <b>33.7</b> |

The “-” symbol in the row corresponding to the standalone method indicates that this method does not belong to either the homogeneous or heterogeneous model structures setting on the devices.

The “-” symbol in the rows corresponding to the FedLoRA and FedAP methods indicates that these methods are not applicable for evaluating server performance, as LLMs cannot be deployed on the server under their experimental settings.

sample accompanied by a corresponding instruction. MMLU is organized as a multiple-choice QA dataset spanning 57 domains.

**Data Partition** In this experiment, the local datasets are sampled from a Dirichlet distribution  $Dir(\lambda)$  to emulate domain heterogeneity among devices [20]. The parameter  $\lambda$  controls the degree of data domain skewness (DDS). Specifically, as  $\lambda \rightarrow 0$ , the local datasets on each device become increasingly biased toward a single domain. The server’s local dataset is uniformly sampled from the global dataset. Each local dataset contains 1,000 samples, of which 80% are used for training on both the devices and the server [41].

**Baselines** We conduct a comparative analysis of Co-PLMs against the following baselines:

- **Standalone** allows both edge devices and the cloud server to independently fine-tune their respective language models using local datasets, without involving any form of collaborative training.
- **FedLoRA** [44] applies low-rank adaptation for parameter-efficient fine-tuning, with the rank  $r$  set to 8. All edge devices deploy language models with the same architecture and communicate with the cloud server solely by transmitting the

updated low-rank adaptation matrices. The server aggregates the received models using simple averaging.

- **FedAP** [15] inserts adapters into both the multi-head attention module and the feed-forward network within each Transformer layer of the language models deployed on edge devices. During the training process, the edge devices update only the adapters while keeping the main model frozen. The updated adapters are then uploaded to the cloud server, where model aggregation is performed through averaging.
- **FedCoLLM** [9] introduces lightweight LoRA modules for mutual knowledge transfer between server-based LLM and on-device SLMs, and employs secure aggregation to preserve privacy during the transfer process.
- **FedMKT** [10] employs a federated mutual knowledge transfer framework that enables selective distillation between the server-side LLM and device-side SLMs, wherein edge devices transmit their output logits and training losses to facilitate bidirectional knowledge exchange.

**Evaluation Metrics** For the QA task, we evaluate the performance of language models using Rouge-L and Exact Match (EM).

- **Rouge-L** measures the overlap of the longest common sub-sequence between the generated and reference answers, effectively capturing both content similarity and fluency.
- **Exact Match** calculates the percentage of generated answers that exactly match the reference answers, reflecting the model's ability to produce fully correct responses.

## 5.2 Performance Analysis

We simulate varying levels of domain heterogeneity in the cloud-edge system by setting different values of  $\lambda \in \{0.1, 1\}$  for DDS. All experiments are conducted five times with different random seeds, and the average results are reported in Table 1. We analyze Co-PLMs under two experimental settings: homogeneous and heterogeneous model structures on the devices. Firstly, we can observe that our framework consistently outperforms the Standalone approach under both homogeneous and heterogeneous model structure settings on the devices, achieving performance gains of 0.30% to 12.50% in Rouge-L and 1.40% to 13.10% in EM across different language models. This indicates that collaborative learning enhances overall model performance, even under scenarios with varying levels of domain heterogeneity.

For the setting of homogeneous model structure on the devices, methods are evaluated under the configuration where edge devices are deployed with the same SLM architecture, and the performance results from identical devices are reported in Table 1. We conduct experiments using three representative language models, Bloom-1.1B, LLaMA2-1.3B, and Qwen2.5-1.5B, across FedLoRA, FedAP, and Co-PLMs. It is worth noting that FedLoRA and FedAP are not applicable for evaluating server performance, as LLMs cannot be deployed on the server under their experimental settings. In contrast, Co-PLMs deploys GPT-J-6B on the server for evaluation. As shown in Table 1, Co-PLMs consistently outperforms FedLoRA and FedAP across different deployed language models, with performance gains of 0.40% to 26.10% in Rouge-L and 0.20% to 17.30% in EM. On average, Co-PLMs achieves gains in 2.38%, 2.55% and 2.83% in Rouge-L and 2.35%, 2.45% and 2.68% in EM compared to FedLoRA on Bloom-1.1B, LLaMA2-1.3B and Qwen2.5-1.5B, respectively. Similarly, Co-PLMs demonstrates average improvements of 5.90%, 18.78% and 5.53% in Rouge-L and 4.98%, 12.15% and 5.10% in EM, compared to FedAP.

To evaluate the performance of Co-PLMs under the setting of heterogeneous model structures on the devices, we configure the experimental setup such that the edge devices are deployed with different SLM architectures. Specifically, Device-1 is deployed with Bloom-1.1B, Device-2 with LLaMA2-1.3B, Device-3 with Qwen2.5-1.5B and server with GPT-J-6B. As presented in Table 1, Co-PLMs significantly outperforms FedCoLLM and FedMKT across all edge devices and the cloud server, achieving performance gains ranging from 0.40% to 12.80%. Specifically, Co-PLMs achieve performance gains of 0.70% to 11.90% and 0.40% to 9.50% in Rouge-L and 1.60% to 12.80% and 0.90% to 7.40% in EM compared to FedCoLLM and FedMKT. On average, Co-PLMs achieves an improvement of 5.48%, 3.45% and 7.05% in Rouge-L and 5.50%, 4.53% and 9.93% in EM compared to FedCoLLM on Device-1, Device-2 and Device-3, respectively. Similarly, Co-PLMs demonstrates average improvements of 3.53%, 2.58% and 4.50% in Rouge-L and 3.43%, 2.85% and 5.60% in

EM compared with FedMKT. Additionally, experimental results indicate that heterogeneous SLM settings outperform homogeneous ones, achieving average gains of 4.07% in ROUGE-L and 2.70% in EM. These improvements are largely attributable to the incorporation of a server-based LLM. In particular, the homogeneous FL baselines (FedLoRA and FedAp) do not include a server-based LLM, whereas the heterogeneous FL baselines (FedCoLLM and FedMKT) explicitly leverage such a component. Furthermore, Table 1 demonstrates that our framework under the homogeneous setting still outperforms FedLoRA and FedAp, yielding average ROUGE-L improvements of 2.58% and 10.06%, respectively, due to the integration of the server-based LLM within Co-PLMs.

It's worth noting that the performance across all methods except Standalone under the setting of  $\lambda = 0.1$  is lower than that under  $\lambda = 1$ , with an average drop of 7.40% in Rouge-L and 3.90% in EM. This decline is due to the domain distribution of the datasets, which increases heterogeneity and makes it more challenging for the models to learn effectively. Specifically, on the SNI dataset, when  $\lambda = 1$ , Co-PLMs under the setting of heterogeneous model structures on the devices obtains average improvements over Standalone, by 1.97%. However, when  $\lambda = 0.1$ , the average of improvement is reduced to 1.59%, indicating the sensitivity of collaborative training to domain heterogeneity.

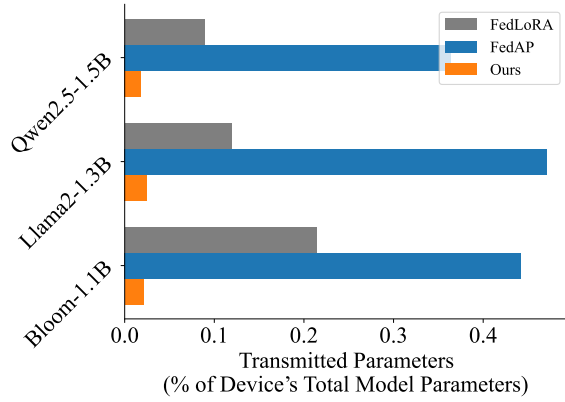
## 5.3 Communication Overhead Analysis

To assess communication efficiency, we compare different baselines in terms of the proportion of transmitted parameters relative to the total number of model parameters on the devices. In the setting of homogeneous model structures at the devices, all devices deploy SLMs with the same architecture and participate in the collaborative training process with an identical total number of model parameters. The communication overhead depends on the communication between the edge devices and the cloud server across different methods. As shown in Figure 3 (a), Co-PLMs transmits only the tunable parameters of DPM between the cloud server and devices, resulting in an average parameter transmission ratio of 0.02%. In contrast, FedLoRa transfers low-rank adaptation matrices and FedAP transmits inserted adapter parameters, both of which require updating and communicating larger portions of the model, ranging from 0.08% to 0.21%, 0.36% to 0.47% respectively.

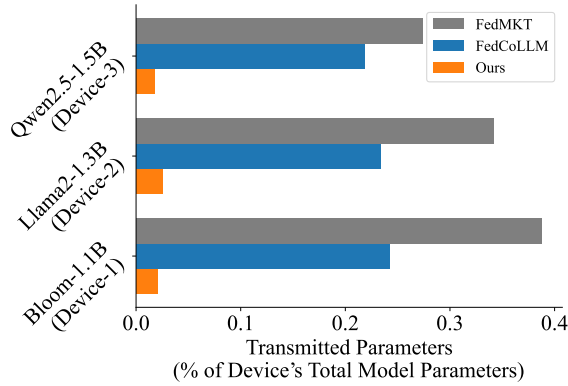
As illustrated in Figure 3 (b), in FedCoLLM, each device transmits its locally trained LoRA modules to the server for model aggregation, which corresponds to a parameter transmission ratio of about 0.21% to 0.24%. Similarly in FedMKT, each device transmits its output logits on the dataset to the server-side LLM for transferring knowledge, the cloud server then sends its logits back to devices for SLM updates. Through bidirectional knowledge transfer, approximately 0.27%–0.38% of parameters are transmitted, which is substantially higher than that required by our framework. In summary, Co-PLMs achieves significantly lower communication overhead while maintaining competitive model performance.

## 5.4 Ablation Study

To validate the effectiveness of the key components in Co-PLMs, we design two variants and compare their performance against



(a) The setting of homogeneous model structures on the devices.



(b) The setting of heterogeneous model structures on the devices.

Figure 3: Communication overheads under different settings.

Table 2: Performance of Co-PLMs for ablation study

| Method    | SNI                   |      |                        |      |                         |      |                   |      |
|-----------|-----------------------|------|------------------------|------|-------------------------|------|-------------------|------|
|           | Bloom-1.1B (Device-1) |      | Llama2-1.3B (Device-2) |      | Qwen2.5-1.5B (Device-3) |      | GPT-J-6B (Server) |      |
|           | Rouge-L               | EM   | Rouge-L                | EM   | Rouge-L                 | EM   | Rouge-L           | EM   |
| Ours      | 42.5                  | 31.7 | 41.8                   | 30.2 | 54.3                    | 39.2 | 59.2              | 39.5 |
| -w/o DST  | 37.8                  | 29.1 | 36.2                   | 27.4 | 52.6                    | 31.7 | 57.4              | 37.8 |
| -w/o SAML | 40.1                  | 29.4 | 38.9                   | 26.7 | 50.1                    | 32.5 | 53.5              | 37.3 |

| Method    | MMLU                  |      |                        |      |                         |      |                   |      |
|-----------|-----------------------|------|------------------------|------|-------------------------|------|-------------------|------|
|           | Bloom-1.1B (Device-1) |      | Llama2-1.3B (Device-2) |      | Qwen2.5-1.5B (Device-3) |      | GPT-J-6B (Server) |      |
|           | Rouge-L               | EM   | Rouge-L                | EM   | Rouge-L                 | EM   | Rouge-L           | EM   |
| Ours      | 44.3                  | 31.0 | 52.9                   | 28.5 | 57.2                    | 39.4 | 60.4              | 38.0 |
| -w/o DST  | 41.4                  | 27.9 | 47.8                   | 24.3 | 47.7                    | 36.1 | 59.8              | 36.9 |
| -w/o SAML | 42.7                  | 28.6 | 50.5                   | 24.9 | 51.2                    | 31.7 | 56.8              | 36.6 |

Co-PLMs on our evaluation metrics, as illustrated in Table 2. Concretely, we investigate the contributions of the DST process and the SAML by removing each component individually and evaluating their impact on model performance across both datasets.

Co-PLMs w/o DST variant is constructed by removing the domain-aware adapters from each Transformer layer of the DPM. In this variant, the model is trained without explicitly preserving domain-dependent knowledge, meaning that local domain-specific patterns are less effectively captured. The results show that Co-PLMs outperforms Co-PLMs w/o DST on both datasets, by margins of

2.90%–4.70%, 5.10%–5.60%, 1.70%–9.50% and 0.60%–1.80% across edge devices and the cloud server. This performance decline demonstrates that the DST process is crucial for mitigating the negative impact of domain heterogeneity and enabling each device model to retain its local domain knowledge.

Co-PLMs w/o SAML variant is designed by omitting the structure-agnostic mutual learning process at the cloud server, such that the server-side LLM is not involved, and only the tunable parameters of DPMs transmitted from each device are aggregated at the cloud server. The table displays that Co-PLMs surpasses by an average of 2.00%, 2.70%, 5.10%, and 4.65% over the w/o SAML variant. These observations confirm that SAML enables the server-side LLM to actively participate in the collaborative learning process, transferring knowledge back and forth between the cloud server and edge devices, which not only allows the cloud server to leverage insights from multiple heterogeneous client models, but also facilitates each device in improving its local SLM's performance, enhancing both generalization and stability under resource-constrained cloud-edge system.

## 6 Conclusion

In this paper, we propose a novel co-tuning framework for collaborative training of large and small language models within cloud-edge systems, aiming to enhance model performance across edge devices and the cloud server while preserving data privacy throughout the collaborative learning process. At the start of the collaborative learning process, the DPM is distilled from the server-based LLM and distributed to all devices, where domain adapters are subsequently integrated. In each collaborative training round, every device locally fine-tunes its DPM through DST to retain domain-specific knowledge and performs SAML with its local SLM to enable bidirectional knowledge exchange. The tunable parameters of all device-side DPMs are then uploaded to the server and aggregated to update the server-based DPM. The server further conducts SAML between the updated DPM and the LLM to refine the LLM. Finally, the server redistributes the updated tunable parameters of the DPM to all devices, thereby propagating knowledge distilled from the LLM. We conduct comprehensive experiments on two datasets and compare our framework with five state-of-the-art baselines to demonstrate its effectiveness. The results show that the proposed framework consistently outperforms existing approaches under the settings of homogeneous and heterogeneous model structures at the devices throughout the collaborative learning process. In future work, we plan to investigate an optimal collaborative training process that explicitly accounts for energy costs in resource-constrained edge-cloud systems, with the goal of further improving the energy efficiency of the proposed framework.

## 7 Acknowledgment

This work is partially supported by National Natural Science Foundation of China under Grants 62502345 and 62441237, and is partially supported by Natural Science Foundation of Shanghai under Grant 25ZR1402511.

## References

- [1] Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. 2021. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. In *the 59th*

- Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*. 7319–7328.
- [2] Fenglong Cai, Dong Yuan, Zhe Yang, and Lizhen Cui. 2024. Edge-llm: A Collaborative Framework for Large Language Model Serving in Edge Computing. In *IEEE International Conference on Web Services*. 799–809.
  - [3] Yi Chen, JiaHao Zhao, and HaoHao Han. 2025. A Survey on Collaborative Mechanisms between Large and Small Language Models. *arXiv preprint arXiv:2505.07460* (2025).
  - [4] Kewei Cheng, Tao Fan, Yilun Jin, Yang Liu, Tianjian Chen, Dimitrios Papadopoulos, and Qiang Yang. 2021. Secureboost: A Lossless Federated Learning Framework. *IEEE Intelligent Systems* 36, 6 (2021), 87–98.
  - [5] Xiangxiang Chu, Limeng Qiao, Xinyu Zhang, Shuang Xu, Fei Wei, Yang Yang, Xiaofei Sun, Yiming Hu, Xinyang Lin, Bo Zhang, et al. 2024. Mobilevlm v2: Faster and Stronger Baseline for Vision Language Model. *arXiv preprint arXiv:2402.03766* (2024).
  - [6] Thomas M Cover. 1999. *Elements of Information Theory*. John Wiley & Sons.
  - [7] Yongheng Deng, Ziqing Qiao, Ye Zhang, Zhenya Ma, Yang Liu, and Ju Ren. 2025. CrossLM: A Data-Free Collaborative Fine-Tuning Framework for Large and Small Language Models. In *the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 124–137.
  - [8] Ning Ding, Yehui Tang, Zhongqian Fu, Chao Xu, Kai Han, and Yunhe Wang. 2025. GPT4Image: Large Pre-trained Models Help Vision Models Learn Better on Perception Task. In *Companion Proceedings of the ACM on Web Conference*. 2056–2065.
  - [9] Tao Fan, Yan Kang, Guoqiang Ma, Lixin Fan, Kai Chen, and Qiang Yang. 2024. FedCoLLM: A Parameter-Efficient Federated Co-tuning Framework for Large and Small Language Models. *arXiv preprint arXiv:2411.11707* (2024).
  - [10] Tao Fan, Guoqiang Ma, Yan Kang, Hanlin Gu, Yuanfeng Song, Lixin Fan, Kai Chen, and Qiang Yang. 2025. FedMKT: Federated Mutual Knowledge Transfer for Large and Small Language Models. In *the 31st International Conference on Computational Linguistics*. 243–255.
  - [11] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. MiniLLM: Knowledge Distillation of Large Language Models. In *International Conference on Learning Representations*. 1–24.
  - [12] Zixu Hao, Huiqiang Jiang, Shiqi Jiang, Ju Ren, and Ting Cao. 2024. Hybrid SLM and LLM for Edge-Cloud Collaborative Inference. In *the Workshop on Edge and Mobile Foundation Models*. 36–41.
  - [13] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*. 1–27.
  - [14] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear units (GELUs). *arXiv preprint arXiv:1606.08415* (2016).
  - [15] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient Transfer Learning for NLP. In *International Conference on Machine Learning*. 2790–2799.
  - [16] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. 1–13.
  - [17] Cong Jiang, Fangzhi Zhu, Xiaowei Chen, Junxiong Zhu, Bo Zheng, Yifeng Wang, and Zheng Zhang. 2025. DMLoRA: Dynamic Multi-Subspace Low-Rank Adaptation. In *the ACM on Web Conference*. 1038–1042.
  - [18] Hongpeng Jin and Yanzhao Wu. 2025. Ce-CoLLM: Efficient and Adaptive Large Language Models through Cloud-Edge Collaboration. In *IEEE International Conference on Web Services*. 316–323.
  - [19] Yuze Liu, Tingjie Liu, Tiejue Zhang, Youhua Xia, Jinze Wang, Zhishu Shen, Jiong Jin, Zhijun Ding, and Fei Richard Yu. 2025. GRL-Prompt: Towards Prompts Optimization via Graph-Empowered Reinforcement Learning Using LLMs' Feedback. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. 426–438.
  - [20] Yuze Liu, Tiejue Zhang, Zhishu Shen, Libing Wu, Shiping Chen, and Jiong Jin. 2026. Towards Heterogeneity-Aware and Energy-Efficient Topology Optimization for Decentralized Federated Learning in Edge Environment. *IEEE Transactions on Mobile Computing* (2026), 1–15.
  - [21] Ao Ma, Zhiyuan Li, Zhuonan Liang, Tiancheng Gu, Jianan Fan, Jietao Long, Henning Müller, and Weidong Cai. 2025. Seek Inner: LLM-Enhanced Information Mining for Medical Visual Question Answering. In *the ACM on Web Conference*. 2297–2305.
  - [22] Vince Nguyen, Vidya Dhope, Hieu Huynh, Hiba Bouhlal, Anusha Annengala, Gian Luca Scoccia, Matias Martinez, Vincenzo Stoico, and Ivano Malavolta. 2025. On-Device or Remote? On the Energy Efficiency of Fetching LLM-Generated Content. In *IEEE/ACM 4th International Conference on AI Engineering—Software Engineering for AI*. 72–82.
  - [23] Z Peng, X Fan, Y Chen, Z Wang, S Pan, C Wen, R Zhang, and C Wang. 2024. FedPFT: Federated Proxy Fine-Tuning of Foundation Models. In *IJCAI-24: Thirty-Third International Joint Conference on Artificial Intelligence*. 1–13.
  - [24] Partha Pratim Ray and Mohan Pratap Pradhan. 2024. LLMEdge: A Novel Framework for Localized LLM Inference at Resource Constrained Edge. In *International Conference on IoT Based Control Networks and Intelligent Systems*. 1–8.
  - [25] Junhong Shen, Neil Tenenholz, James Brian Hall, David Alvarez-Melis, and Nicolo Fusi. 2024. Tag-LLM: Repurposing General-Purpose LLMs for Specialized Domains. In *International Conference on Machine Learning*. 44759–44773.
  - [26] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Riviere, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open Models Based on Gemini Research and Technology. *arXiv preprint arXiv:2403.08295* (2024).
  - [27] Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model.
  - [28] Kaibin Wang, Qiang He, Feifei Chen, Chunyang Chen, Faliang Huang, Hai Jin, and Yun Yang. 2023. Flexified: Personalized Federated Learning for Edge Clients with Heterogeneous Model Architectures. In *the ACM Web Conference*. 2979–2990.
  - [29] Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, et al. 2022. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. In *the Conference on Empirical Methods in Natural Language Processing*. 5085–5109.
  - [30] BigScience Workshop, Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, et al. 2022. Bloom: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv preprint arXiv:2211.05100* (2022).
  - [31] Xinghao Wu, Xuefeng Liu, Jianwei Niu, Haolin Wang, Shaojie Tang, and Guogang Zhu. 2024. FedLoRA: When Personalized Federated Learning Meets Low-Rank Adaptation. In *International Conference on Learning Representations*. 1–14.
  - [32] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2023. Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning. In *Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@NeurIPS)*. 1–22.
  - [33] Youcheng Yan, Jinshuo Liu, Donghong Ji, Jinguang Gu, Ahmed Abubakar Aliyu, Xinyan Wang, and Jeff Z Pan. 2024. Collaborate SLM and LLM with Latent Answers for Event Detection. *Knowledge-Based Systems* 305 (2024), 112684.
  - [34] An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiaodong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, et al. 2025. Qwen2.5-1m Technical Report. *arXiv preprint arXiv:2501.15383* (2025).
  - [35] Xingke Yang, Liang Li, Sicong Li, Liwei Guan, Hao Wang, Xiaoqi Qi, Jiang Liu, Xin Fu, and Miao Pan. 2025. Fed MobilLM: Efficient Federated LLM Fine-Tuning over Heterogeneous Mobile Devices via Server Assisted Side-Tuning. *arXiv preprint arXiv:2508.06765* (2025).
  - [36] Zheming Yang, Yuanhao Yang, Chang Zhao, Qi Guo, Wenkai He, and Wen Ji. 2024. PerLLM: Personalized Inference Scheduling with Edge-Cloud Collaboration for Diverse LLM Services. *arXiv preprint arXiv:2405.14636* (2024).
  - [37] Liping Yi, Han Yu, Gang Wang, Xiaoguang Liu, and Xiaoxiao Li. 2023. pFedLoRA: Model-heterogeneous Personalized Federated Learning with LoRA Tuning. *arXiv preprint arXiv:2310.13283* (2023).
  - [38] Zhongzhi Yu, Zheng Wang, Yuhao Li, Ruijie Gao, Xiaoya Zhou, Sreenidhi Reddy Bommur, Yang Zhao, and Yingyan Lin. 2024. EDGE-LLM: Enabling Efficient Large Language Model Adaptation on Edge Devices via Unified Compression and Adaptive Layer Voting. In *ACM/IEEE Design Automation Conference*. 1–6.
  - [39] Chunjiang Zhang, Gaoyang Shan, Byeong-hee Roh, and Jun Jiang. 2025. FM2 Learning: LLM-Based Federated Multi-Task Multi-Domain Learning for Consumer Electronics and IoT Enhancement. *IEEE Transactions on Consumer Electronics* 71 (2025), 11977–11988.
  - [40] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An Open-source Small Language Model. *arXiv preprint arXiv:2401.02385* (2024).
  - [41] Tiejue Zhang, Yuze Liu, Zhishu Shen, Rui Xu, Xin Chen, Xiaowei Huang, and Xi Zheng. 2023. An Adaptive Federated Relevance Framework for Spatial–Temporal Graph Learning. *IEEE Transactions on Artificial Intelligence* 5, 5 (2023), 2227–2240.
  - [42] Tiejue Zhang, Rui Xu, Jianping Zhang, Yuze Liu, Xin Chen, Jun Yin, and Xi Zheng. 2024. DSHGT: Dual-Supervisors Heterogeneous Graph Transformer—A Pioneer Study of Using Heterogeneous Graph Learning for Detecting Software Vulnerabilities. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–31.
  - [43] Ying Zhang, Tao Xiang, Timothy M Hospedales, and Huchuan Lu. 2018. Deep Mutual Learning. In *the IEEE Conference on Computer Vision and Pattern Recognition*. 4320–4328.
  - [44] Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu. 2023. FedPETuning: When Federated Learning Meets the Parameter-Efficient Tuning Methods of Pre-trained Language Models. In *Annual Meeting of the Association of Computational Linguistics*. 9963–9977.
  - [45] Haodong Zhao, Wei Du, Fangqi Li, Peixuan Li, and Gongshen Liu. 2023. Fed-Prompt: Communication-Efficient and Privacy-Preserving Prompt Tuning in Federated Learning. In *IEEE International Conference on Acoustics, Speech and Signal Processing*. 1–5.